

The `MusicianOut` serializer defined in the previous exercise includes the `Musician` ID. This ID should not be included in the set of fields used to update an object, as the ID is provided as part of the URL. This means you'll need a different serializer for this endpoint. The following listing shows this variation.

Listing E.49 Serializers for the Band and Musician objects

```
# RiffMates/bands/api.py
...
from ninja import ModelSchema
...
from bands.models import Musician
...
class MusicianIn(ModelSchema):
    class Meta:
        model = Musician
        fields = ["first_name", "last_name", "birth", "description",]
```

The API endpoint view uses the PUT action to do an update requiring all field information to be set. The response of the PUT call is the modified `Musician` object. This endpoint requires authentication, which, in this case, simply means specifying the `APIKeyHeader` object used throughout chapter 11.

Inside the view, the code loops through each of the fields in the serializer, applying the new value to the `Musician` object. Once complete, the object gets saved and returned. The following listing shows the code required.

Listing E.50 Serializers for the Band and Musician objects

```
# RiffMates/bands/api.py
...
from api_auth import api_key  ← Import the APIKeyHeader object
                               from the shared module.
...
@router.put(
    "/musician/{musician_id}/",
    response=MusicianOut,      ← The response from a PUT
    auth=api_key               action is a serialization of the
                               modified Musician object.
)                               ← The auth argument takes
                               an authentication object,
                               in this case the shared-
                               key mechanism.

def update_musician(request, musician_id,
    payload:MusicianIn):
    musician = get_object_or_404(Musician,
    id=musician_id)             ← Fetch the
    for key, value in payload.dict().items():  Musician
    setattr(musician, key, value)             object for
                                              the given ID.

    musician.save()             ← Save the changes to the
    return musician              Musician object, and return
                               it so that Ninja can serialize
                               the new values.

    ← Loop through the payload,
    and for each key-value pair,
    update the corresponding
    field in the Musician object.
```